

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	CH.POORNA TEJA	Reg. No.:	192472149
Section / Batch:	2024	Date:	22-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT2- Industry Problem Solving Task

Language: Python C C++ Java

QUESTIONS

Q1

Smart Manufacturing
Production Scheduling
Optimization (Job Scheduling +
DP)

100 marks

1/1 answered

Q1 Smart Manufacturing Production Scheduling Optimization (Job Scheduling + DP)

100 marks

A smart factory must schedule multiple production jobs across limited machines to minimize total completion time and delays. Analyze how job scheduling can be modeled using optimization algorithms with constraints such as machine availability and job priorities. Identify challenges like dynamic job arrivals and resource conflicts. Design a solution using Dynamic Programming or Greedy strategies to optimize throughput. Discuss feasibility in real-time manufacturing systems. Evaluate time and space complexity and interpret results in an industrial automation context.

Your Code

```
1 # Smart Manufacturing Production Scheduling
2 # Greedy Job Scheduling
3
4 n = int(input("Enter number of jobs: "))
5 m = int(input("Enter number of machines: "))
6
7 jobs = []
8
9 print("Enter processing time and priority for each job:")
10 for i in range(n):
11     p, priority = map(int, input().split())
12     jobs.append((p, priority, i + 1))
13
14 # Sort by priority (higher priority first)
15 # If priority is same, shorter job first
16 jobs.sort(key=lambda x: (-x[1], x[0]))
17
18 # Available time of each machine
19 machine_time = [0] * m
20
21 schedule = [[] for _ in range(m)]
22
23 # Assign jobs
24 for p, priority, job in jobs:
25     # Find machine available earliest
26     machine = machine_time.index(min(machine_time))
27
```

Input

```
5
2
4 2
3 5
```

Output

```
Enter number of jobs: Enter
number of machines: Enter
processing time and priority for
each job:
Job Schedule:
```

Algorithm: Smart Manufacturing Production Scheduling

Step 1: Start.

Step 2: Read the number of jobs n and number of machines m .

Step 3: For each job, read:

- Processing time
- Priority
- Job number

Step 4: Store all jobs in a list.

Step 5: Sort the jobs:

- Higher priority jobs first.
- If priorities are equal, shorter processing time first.

Step 6: Initialize the available time of every machine to 0.

Step 7: For each job:

1. Find the machine with the **minimum available time**.

2. Assign the job to that machine.
3. Set the start time as the machine's available time.
4. Calculate:
Finish Time = Start Time + Processing Time
5. Update the machine's available time.

Step 8: Display the schedule of all machines with job numbers, start times, and finish times.

Step 9: Calculate and display:

- Total completion time
- Overall completion time

Step 10: Stop.

Complexity

- **Sorting:** $O(n \log n)$
- **Job assignment:** $O(n \times m)$
- **Overall Time Complexity:** $O(n \log n + n \times m)$
- **Space Complexity:** $O(n + m)$

Main idea: Use a **Greedy strategy** by giving priority to important jobs and assigning each job to the machine that becomes available earliest.